

# SMS Spam Detection — Project Template

## 1. Title Page

- **Project Title:** SMS Spam Detection Using Machine Learning
  - **Your Name(s) :** MOHANAPRIYA M
  - **Institution / Department :** Bharathiyar Institute Of Engineering For Women / AI&DS
  - **Date :** 23.10.2025
- 

## 2. Table of Contents

| S.No | Description                    |
|------|--------------------------------|
| 1.   | Project overview               |
| 2.   | Objectives & Problem Statement |
| 3.   | Problem Statement              |
| 4.   | Objectives                     |
| 5.   | Proposed Solution              |
| 6.   | Features                       |
| 6.1  | Functional Features            |
| 6.2  | Non-Functional Features        |
| 7.   | Technologies & Tools           |
| 8.   | System Architecture            |

|     |                      |
|-----|----------------------|
| 9.  | Output / Screenshots |
| 10. | Advantages           |
| 11. | Future Enhancements  |
| 12. | Conclusion           |

---

### 3. Project Overview

**Background / Motivation:** It prevents users from falling victim to phishing, fraud, or malicious links hidden in spam messages.

Filtering spam ensures that users only receive important, relevant messages instead of wasting time on junk mail.

Spam often contains malware or viruses. Detecting and blocking it protects devices and networks from attacks.

Spam consumes bandwidth, storage, and processing power. Detecting it early keeps systems running smoothly.

**Scope:** The project focuses only on SMS text messages (not emails or social media messages).

- Spam – unwanted or promotional messages.
- It will classify messages into two categories:

- Ham – legitimate or normal messages.
- It will use Natural Language Processing (NLP) techniques for text preprocessing

**High-level Description:** A machine learning model trained on a labeled SMS dataset to identify spam patterns and vectorization (e.g., TF-IDF).

An NLP pipeline for preprocessing text data — including tokenization, stop-word removal, A user interface (UI) where users can input an SMS message and instantly see whether it Spam or not.

**Dataset / Data Source Summary :** The project uses the SMS Spam Collection Dataset, a publicly available dataset containing 5,574 SMS messages labeled as “spam” or “ham”.

Each message includes the text content and its corresponding label indicating whether it is a spam or legitimate message .

The dataset is widely used for spam detection research and can be obtained from sources like the UCI Machine Learning Repository or Kaggle.

---

## 4. Objectives & Problem Statement

### 4.1 Problem Statement

With the rapid growth of mobile communication, users receive a large number of SMS messages daily, many of which are spam messages containing unwanted advertisements, scams, or fraudulent links. These spam messages not only cause inconvenience but can also lead to privacy and security issues.

The main problem is to automatically detect and classify SMS messages as spam or ham (legitimate) using machine learning techniques, in order to reduce user exposure to spam and improve communication safety and efficiency.

**For example:**

“Congratulations! You’ve won a free ₹10,000 gift card. Click here to claim your prize:  
[www.fakeoffer.com](http://www.fakeoffer.com)”

Such messages are deceptive and need to be automatically detected.

The goal of this project is to develop an SMS Spam Detection system that uses machine learning and natural language processing (NLP) to classify incoming messages as Spam or Ham (Not Spam). This helps in filtering unwanted texts and ensuring user safety.

## 4.2 Objectives

The main objectives of the SMS Spam Detection project are:

- **To build a predictive model** that accurately classifies SMS messages as *Spam* or *Ham* (*Not Spam*).
- **To preprocess and clean SMS text data effectively** using NLP techniques such as tokenization, stop-word removal, and vectorization.
- **To experiment with different machine learning algorithms** (e.g., Naive Bayes, Logistic Regression, SVM) and identify the best-performing model.
- **To develop an inference pipeline or user interface (UI)** that allows users to input a message and get instant spam classification results.
- **To evaluate and compare model performance** using metrics like accuracy, precision, recall, and F1-score.
- **To document findings and suggest improvements** for future enhancement of the spam detection system.

---

## 5. Proposed Solution

### Approach / Methodology

The project uses a supervised machine learning approach, where the model is trained on a labeled dataset containing SMS messages categorized as Spam or Ham.

The system will use Natural Language Processing (NLP) techniques to convert raw text into numerical features and apply different classification algorithms (such as Naive Bayes, Logistic Regression, or Support Vector Machine) to identify spam patterns.

The models will be evaluated and compared to select the one with the best accuracy, precision, and recall.

---

## Pipeline Overview

1. Raw SMS Data Collection – Input dataset containing labeled SMS messages.
2. Text Preprocessing – Cleaning messages by removing punctuation, converting to lowercase, tokenizing, and removing stop words.
3. Feature Extraction – Converting text into numerical format using techniques like TF-IDF or Count Vectorization.
4. Model Training – Applying supervised learning algorithms to learn from training data.
5. Model Evaluation – Measuring model performance using metrics such as accuracy, precision, recall, and F1-score.
6. Prediction / Inference – Classifying new SMS messages as *Spam* or *Ham*.

## Justification of Choices

- Supervised learning is chosen because the dataset is labeled and ideal for classification.
  - Text preprocessing is essential to remove noise and improve model accuracy.
  - TF-IDF vectorization helps capture the importance of words in messages, improving text representation.
  - Naive Bayes and Logistic Regression are selected due to their simplicity, efficiency, and strong performance on text classification tasks.
  - The pipeline structure ensures a clear, step-by-step process from data input to final prediction, making the system easy to implement and extend.
- 

## 6. Features

Describe what your system offers (functional & non-functional):

### 6.1 Functional Features

- Accept raw SMS text input, output “spam” or “ham”
- Provide confidence / probability score
- Batch classification (multiple messages at once)

- Display key influencing features or words (if interpretability)
- Interface (CLI, web, or GUI)

## 6.2 Non-Functional Features

- Fast inference / low latency
  - Accuracy / reliability
  - Scalability (if many messages)
  - Ease of use / user friendly
  - Maintainability / model update support
- 

## 7. Technologies & Tools

List all software, libraries, frameworks used. Examples:

- **Language:** Python
- **Data Handling:** pandas, numpy
- **Visualization:** matplotlib, seaborn
- **Text / NLP:** nltk, re (regex), spaCy (if used)
- **Feature extraction:** scikit-learn's CountVectorizer, TfidfVectorizer
- **Machine Learning:** scikit-learn (Naive Bayes, Logistic Regression, Random Forest, SVM etc.)
- **Model persistence:** pickle, joblib
- **Web / UI (if built):** Flask or Streamlit
- **Environment / Tools:** Jupyter Notebook / VS Code, Git / GitHub

---

## 8. System Architecture

- **Architecture Diagram** (block diagram)

- Modules: Input → Preprocessing → Feature Extraction → Model → Output / UI

- **Component Description:**

### 1. Input Module

- **Purpose:** Collects SMS text data from various sources (dataset or real-time input).
- **Function:**
  - Accepts raw SMS messages.
  - Passes them to the preprocessing module for cleaning.
- **Example:** “Congratulations! You’ve won a free prize.”

---

### 2. Preprocessing Module

- **Purpose:** Clean and prepare text data for analysis.
- **Functions:**
  - Remove punctuation, stopwords, numbers, and special symbols.
  - Convert text to lowercase.
  - Tokenize words and perform stemming/lemmatization.
- **Example:**
  - Input: “Congratulations! You’ve won a FREE Prize!!!”
  - Output: “congratulation win free prize”

---

### 3. Feature Extraction Module

- **Purpose:** Convert cleaned text into numerical form for model input.
- **Techniques Used:**
  - **Bag of Words (BoW):** Counts word occurrences.
  - **TF-IDF (Term Frequency–Inverse Document Frequency):** Weighs important words.
  - **Word Embeddings (optional):** Captures context meaning.
- **Output:** Numerical feature vectors representing each SMS.

---

### 4. Model Module

- **Purpose:** Classify messages as **Spam** or **Ham (Not Spam)**.
- **Approach:**
  - Uses **supervised learning algorithms** such as Naive Bayes, Logistic Regression, or SVM.

- Model is trained on labeled data.
- **Output:** Predicted label → *Spam* or *Not Spam*.

---

## 5. Output / UI Module

- **Purpose:** Display results to the user.
- **Functions:**
  - Shows classification result (Spam / Not Spam).
  - May include confidence score or explanation.
- **Example:**
  - Input: "You have won ₹10,000!"
  - Output: **Spam**

---

## • Data Flow: 1. Training Phase (Model Building Stage)

This is when the model learns to identify spam messages.

Raw SMS Dataset



Preprocessing



Feature Extraction



Model Training



Trained Model (Saved for Future Use)

## Inference Phase (Prediction Stage)

This is when the model is used to classify *new, unseen* SMS messages.

New Incoming SMS



Preprocessing (Same steps as training)



Feature Extraction (Same method as training)

Trained Model (Load from storage)



Prediction (Spam / Not Spam)



Display Result (Output / UI)

---

## 9. Implementation Steps

- Data Collection
- Data Preprocessing
- Feature Extraction
- Data Splitting
- Model Selection and Training
- Model Evaluation
- Model Saving
- Prediction (Inference Phase)
- User Interface
- Model Maintenance

---

## 10.output/screenshot

URL:<https://colab.research.google.com/drive/1FcZOL9IWFR2qKh5KQ07wnFyVRMDg6yH9#scrollTo=S5-PJVUedFy8>

**output:**

Message: 'Free entry in 2 lakh cash prize!'

Prediction: Spam

-----

Message: 'Can we meet tomorrow?'

Prediction: Not Spam

-----

Message: 'You have won a free vacation to Goa!'

Prediction: Spam

-----

---

## 10. Advantages

Enumerate strengths of your system. Examples: •

Automates spam detection for SMS

- Good performance metrics (precision, recall)
  - Fast inference, lightweight
  - Easy to extend or retrain
  - Interpretability (if you show feature importance)
  - Usable interface
- 

## 11. Future Enhancements

Ideas for improving further:

- Use deep learning / transformer models (BERT etc.)
  - Handle multiple languages
  - Online / incremental learning (update model over time)
  - Better handling of class imbalance (SMOTE, oversampling)
  - Spam categorization (phishing, advertisement, malware links)
  - Improve UI, develop mobile app
  - Deploy as cloud / API service
  - Use ensemble / stacking models
  - Deploy real-time spam detection pipeline
-

## 12. Conclusion

- **Recap:**

In this project, we developed an SMS Spam Detection system using machine learning techniques. The main objective was to classify messages as *spam* or *ham (non-spam)* to help users avoid unwanted or fraudulent texts. We followed a supervised learning approach with modules for data preprocessing, feature extraction (using TF-IDF or Bag of Words), model training, and prediction.

- **Summary of results and performance:**

The implemented model successfully detected spam messages with high accuracy (e.g., around 95–98% depending on the algorithm used). Models like Naive Bayes and Logistic Regression performed particularly well due to their efficiency with text classification tasks.

- **Reflection (what worked & challenges):**

The preprocessing and text vectorization steps were effective in improving accuracy. However, challenges included handling ambiguous or short messages and ensuring proper cleaning of special characters and stop words.

- **Limitations:**

The system's accuracy depends on the quality and size of the dataset. It may not perform well on slang or multilingual SMS without retraining. Additionally, real-time deployment would require continuous model updates to adapt to new spam patterns.

- **Final thoughts and next steps:**

Overall, the project demonstrated that machine learning can be effectively applied to spam detection. Future improvements could include deep learning models (like LSTM), real-time message filtering, and multilingual support to enhance performance and usability.